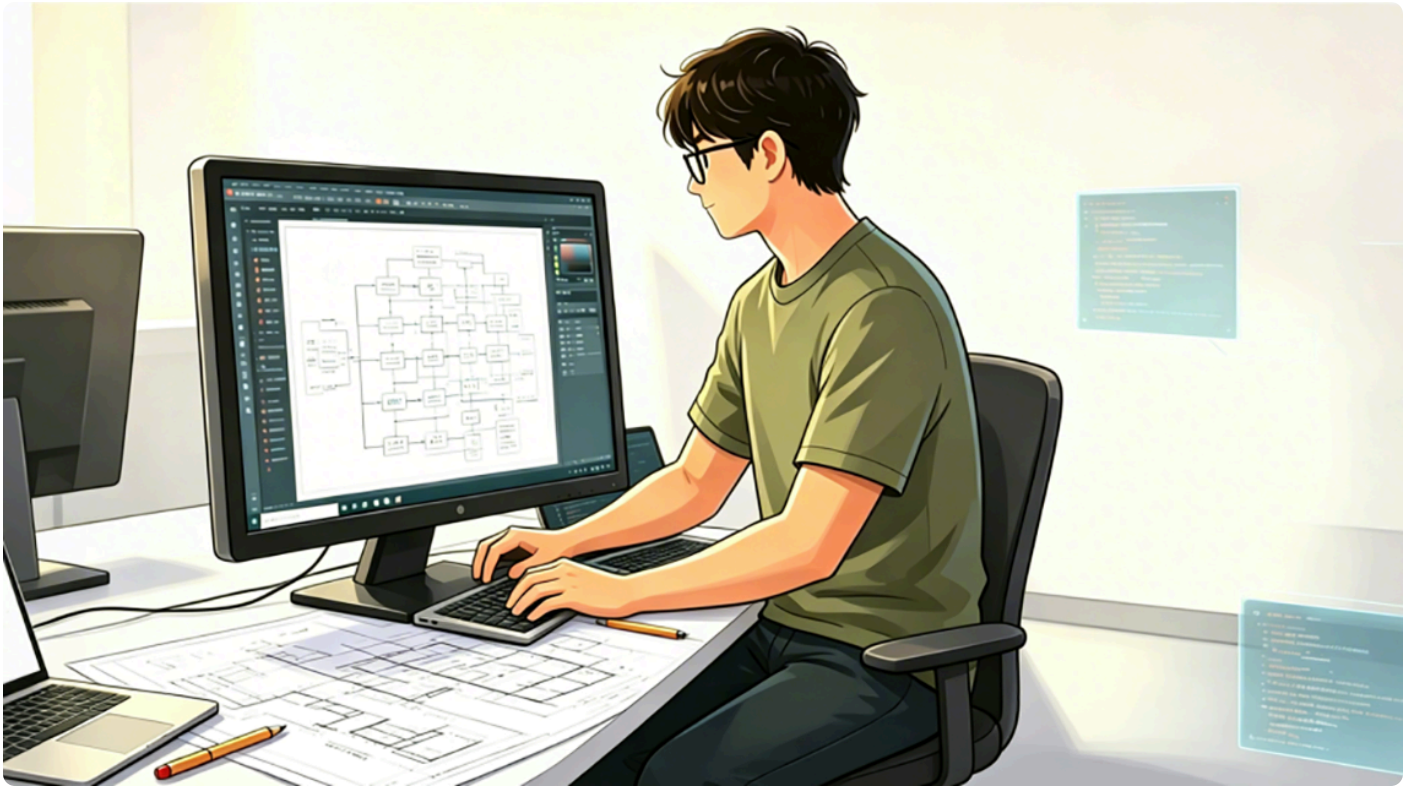
05 | 架构设计（下）：部署架构、性能预判与数据设计

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

上一讲我们定了代码层面的架构，包括模块怎么分、代码怎么组织、外部调用怎么处理。这一讲进入系统层面：当前怎么部署、瓶颈在哪、未来怎么扩展、数据怎么存。

很多工程师觉得部署是上线前的事。但部署架构从第一天就在影响你的设计决策——**缓存放哪、数据库怎么连、服务之间怎么通信，全都取决于你的部署形态。提前想清楚，后面不返工。**

当前部署架构：50 人规模

先把当前的部署全貌画出来。我让 Claude Code 帮我梳理：

> Hify 是模块化单体，技术栈 Spring Boot + Vue + MySQL + Redis + pgvector。目标 50 人内部使用，生产环境用 Docker + K8s 部署。帮我设计当前阶段的部署架构：有哪些组

> 件、请求怎么流转、每个组件的职责是什么。

Claude Code 给的架构总结很清晰:



部署架构概述

当前阶段是标准的单体 + 三个有状态服务的组合。用户请求经 Nginx 分流，静态资源直接返回，API 请求转发给 Spring Boot，Spring Boot 按需读写 MySQL（业务数据）、Redis（缓存会话）、pgvector（向量检索）。K8s 只作为部署壳，不引入服务网格和多副本等复杂机制。

```
浏览器 → Nginx → Spring Boot → MySQL
                              → Redis
                              → pgvector
```

组件职责

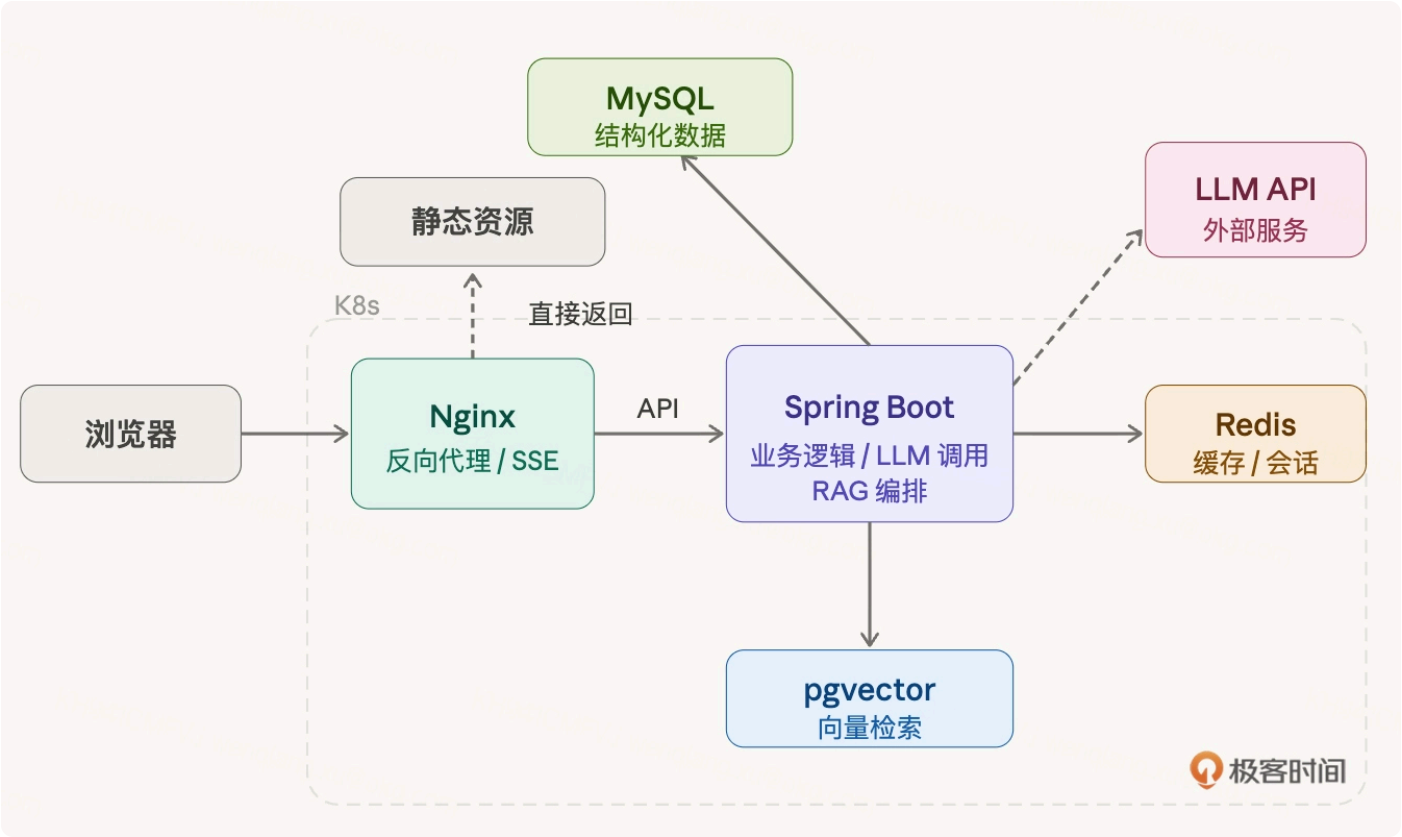
组件	职责
Nginx	静态文件托管、反向代理、SSE 透传（需关闭 proxy_buffering）
Spring Boot	业务逻辑、LLM 调用、RAG 编排、权限校验
MySQL	用户、Agent、Provider、对话日志等结构化数据
Redis	JWT 会话、Provider 配置缓存、Token 用量计数
pgvector	知识库文档向量存储与相似度检索

当前做 vs 暂时跳过

	当前做	跳过理由
副本数	单副本	50人用不到，多副本要处理会话粘连
异步任务	Spring `@Async`	MQ 是过度设计，当前同步 SSE 足够
向量库	pgvector（内嵌 PG）	少维护一个服务，够用再换 Qdrant
可观测	Actuator + 日志	单体服务不需要链路追踪

Claude Code 回答是标准的单体 + 三个有状态服务的组合。用户请求经 Nginx 分流，静态资源直接返回，API 请求转发给 Spring Boot，Spring Boot 按需读写 MySQL（业务数据）、Redis（缓存会话）、pgvector（向量检索）。K8s 只作为部署壳，不引入服务网格和多副本等复杂机制。

请求流转链路：



各组件职责：

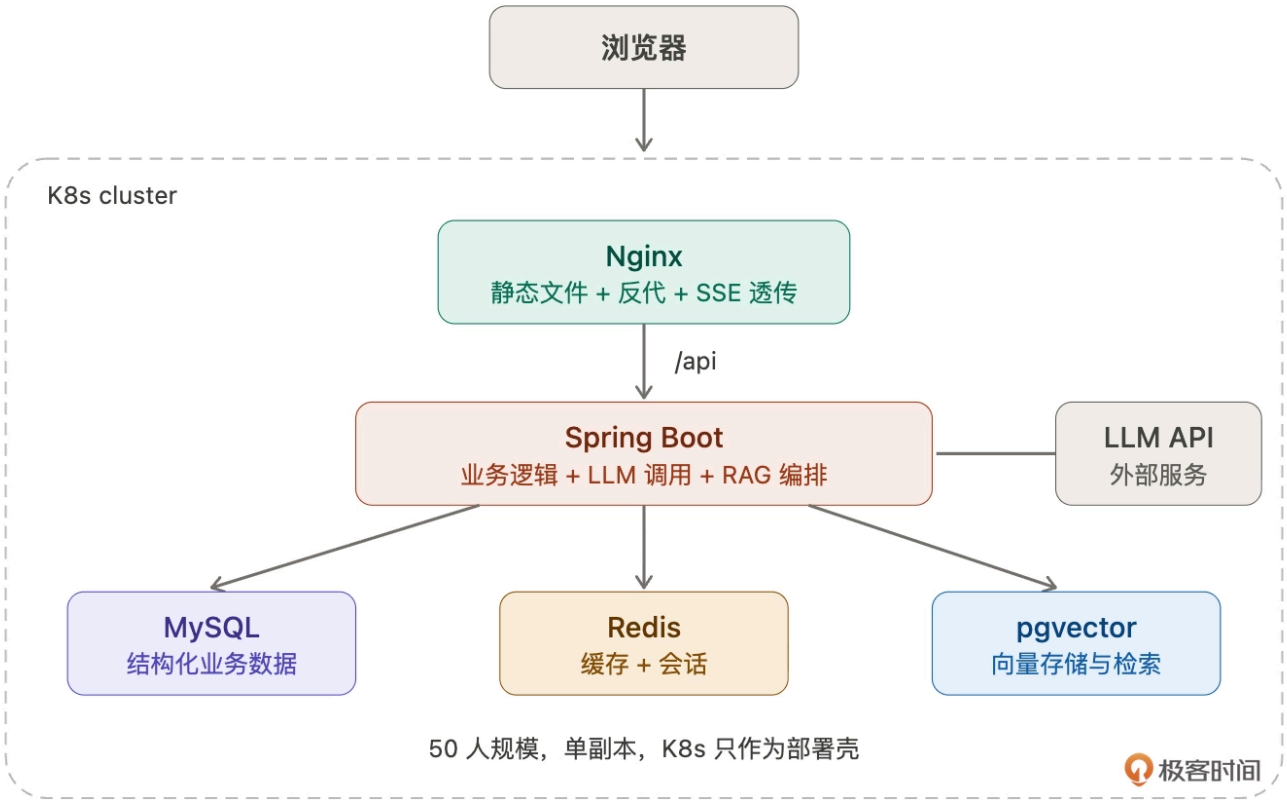
组件	职责
Nginx	静态文件托管、反向代理、SSE 透传（需关闭 proxy_buffering）
Spring Boot	业务逻辑、LLM 调用、RAG 编排
MySQL	Provider、Agent、对话日志等结构化数据
Redis	配置缓存、会话上下文、Token 用量计数
pgvector	知识库文档向量存储与相似度检索

Claude Code 还给了一个很有价值的对比——当前阶段该做什么、暂时跳过什么：

	当前做	暂时跳过	理由
副本数	单副本	多副本	50 人用不到，多副本要处理会话粘连
异步任务	Spring @Async	消息队列	MQ 是过度设计，当前同步 + @Async 足够
向量库	pgvector（内嵌 PG）	独立 Qdrant/Milvus	少维护一个服务，数据量小够用
可观测	Actuator + 日志	Prometheus + 链路追踪	单体服务不需要链路追踪

第二张表是我觉得 Claude Code 这次分析最有价值的部分——它不只告诉你“现在长什么样”，还帮你划了一条“现在做”和“以后做”的线。每个“暂时跳过”都有明确理由，不是偷懒，是一期确实不需要。等触发条件到了（用户量上来、数据量变大），再逐步加。

在这一步，Claude Code 给了一个挺不错的架构，没什么大问题，基本就是我们要的。



但是其实还是需要你的积累和判断，因为这个部署架构较为简单，如果是更复杂的架构，就需要我们去调整，也就是需要积累。无脑让 AI 完全实现一个应用，我认为当前阶段还是达不到的。

这里扩展一个点：**我觉得以后程序员的价值在于思考、拆解、边界判断等架构层面的能力。说实话，这个要求比之前高多了。也就呼应了第 01 讲，我们需要是一个架构师，不应该是一个写代码的程序员。**

性能瓶颈预判

大部分工程师不会提前想性能瓶颈，等出了问题再救。但你作为架构师，应该在设计阶段就知道瓶颈可能在哪。即使一期不处理，也要做到心里有数。

这一步特别适合让 Claude Code 帮你分析：

> 基于 Hify 当前的部署架构（Nginx + Spring Boot + MySQL + Redis + pgvector + 外部 LLM API），帮我分析：这个系统的性能瓶颈可能在哪？按严重程度排序，每个瓶颈给出触发条件和一期是否需要处理。

Claude Code 的输出如下：



瓶颈分析

一期不需要处理任何性能瓶颈。50人规模的真实压力极小，所有瓶颈的触发条件都远超当前规模。唯一需要提前防范的是**外部 LLM API 的超时和费用失控**，这不是性能问题，是稳定性和成本问题。

按严重程度排序

优先级	瓶颈	触发条件	一期处理?
最高	外部 LLM API 响应慢/超时	任何时候，LLM 单次调用 5-30s，50个并发 SSE 连接长期占用线程	要，设超时时间 + 熔断，防止线程堆积
高	RAG 向量检索慢	知识库超过 100万向量，且未建索引	不要，当前文档量不会触发
中	MySQL 慢查询	对话日志表超过百万行，且无分页/索引	不要，建好索引够用半年
中	Spring Boot 线程池耗尽	并发 SSE 连接超过 200（默认线程池上限）	不要，50人峰值约 30 连接
低	Redis 缓存击穿	缓存集中过期 + 瞬间高并发	不要，50人不存在瞬间高并发
低	Nginx 静态资源	未开启 gzip，前端 bundle 过大	建议做，10分钟配置，体验提升明显
可忽略	pgvector 内存占用	向量维度高（1536d）+ 数据量大时全量加载	不要，当前数据量可忽略

一期只做这两件事

事项	原因
LLM 调用设超时 + 线程隔离	唯一能在50人规模下真实触发、影响所有用户的问题
Nginx 开启 gzip	成本极低，首屏加载速度直接可感知

Claude Code 的结论很直接：**一期不需要处理任何性能瓶颈。50 人规模的真实压力极小，所有瓶颈的触发条件都远超当前规模。唯一需要提前防范的是外部 LLM API 的超时和费用失控**——这不是性能问题，是稳定性和成本问题。

它按严重程度排了序：

优先级	瓶颈	触发条件	一期处理?
最高	LLM API 响应慢/超时	任何时候，单次 5-30s，50 个并发 SSE 长期占线程	要，超时 + 熔断 + 线程隔离
高	RAG 向量检索慢	知识库超过 100 万向量且未建索引	不要，当前文档量不会触发
中	MySQL 慢查询	chat_message 表超百万行且无索引	不要，建好索引够用半年
中	Spring Boot 线程池耗尽	并发 SSE 超过 200（默认线程池上限）	不要，50 人峰值约 30 连接
低	Redis 缓存击穿	缓存集中过期 + 瞬间高并发	不要，50 人不存在瞬间高并发
低	Nginx 未开 gzip	前端 bundle 过大	建议做，10 分钟配好，体验提升明显

看这张表你会发现，真正在 50 人规模下能触发的只有第一行——LLM 调用慢且占线程。其他瓶颈的触发条件都是百万级数据或几百人并发，离我们很远。

所以一期只做两件事：

1. **LLM 调用设超时 + 线程隔离 + 熔断**。这是唯一能在 50 人规模下真实触发、影响所有用户的问题。第 04 讲已经设计好了方案（独立线程池、Resilience4j、超时控制），后面实现时落地就行。
2. **Nginx 开启 gzip**。成本极低（一行配置），首屏加载速度直接可感知。

其他瓶颈全部标记“已知但暂不处理”。不是忽略，是时机未到。等触发条件到了（数据量上来、用户量上来），你第一时间知道该动哪里——因为这张表就是你的地图。

这一节的价值不在于解决了什么问题，在于你脑子里有了一张系统“软肋”地图。后面出了性能问题，你不用猜，查表就行。而这张表不需要你自己从零分析，给 Claude Code 你的架构图，它几分钟就能帮你画出来。

扩展架构：几千人规模

一期 50 人，但如果 Hify 做得好，可能要推广到更大范围。架构上不能堵死这条路。但也不能提前演进，过早优化比性能瓶颈本身的破坏力更大。

我让 Claude Code 设计演进路径：

> 如果 Hify 要从 50 人扩展到几千人，当前架构需要怎么演进？帮我设计一个分阶段的扩展路径，每一步的触发条件是什么、改什么、不改什么。

Claude Code 给了三个阶段：

演进路径总览

不要提前演进。每一步都有明确的触发条件，条件不到就不动。过早优化比性能瓶颈本身的破坏力更大。

三个阶段

第一阶段：50 → 500人

项目	内容
触发条件	响应变慢，`docker stats` 显示 CPU/内存持续告警
改什么	Spring Boot 从 1 副本扩到 2-3 副本；MySQL 加读写分离（一主一从）；静态资源上 CDN
不改什么	整体单体架构不动；pgvector 不换；Redis 单节点不动
难点	Spring Boot 多副本要处理 SSE 的连接粘连（用 Redis 做会话共享）

第二阶段：500 → 2000人

项目	内容
触发条件	LLM 调用队列堆积，对话成功率下降；知识库检索 P99 > 2s
改什么	引入 MQ（RocketMQ/Kafka）做 LLM 异步调用削峰；pgvector 迁移到独立 Qdrant；MySQL 分库（按业务模块拆）
不改什么	Spring Boot 仍是单体，不做微服务拆分；Redis 架构不动
难点	LLM 异步化后前端要从 SSE 改成轮询或 WebSocket，是breaking change

第三阶段：2000 → 几千人

项目	内容
触发条件	单个模块的发布频率互相干扰；某一模块（如 Agent 执行）资源消耗远超其他模块
改什么	按模块拆微服务（Agent执行、知识库、对话历史各自独立）；引入 API Gateway 替代 Nginx 路由；Redis 换集群模式
不改什么	MySQL 主从架构不动；Qdrant 不动；K8s 编排层不动
难点	微服务拆分带来分布式事务问题，需要引入 Saga 或 TCC 模式

如上图所示：每个阶段有明确的触发条件，条件不到就不动。我们来总结下：

第一阶段：50 → 500 人

触发条件：响应变慢，docker stats 显示 CPU / 内存持续告警。

改什么：Spring Boot 从 1 副本扩到 2–3 副本；MySQL 加读写分离（一主一从）；静态资源上 CDN。

不改什么：整体单体架构不动，pgvector 不换，Redis 单节点不动。

难点：Spring Boot 多副本要处理 SSE 的连接粘连——用户的流式对话不能被负载均衡随机分发到不同实例。解决方案是用 Redis 做会话共享。

第二阶段：500 → 2000 人

触发条件：LLM 调用队列堆积，对话成功率下降；知识库检索 P99 > 2s。

改什么：引入消息队列（RocketMQ/Kafka）做 LLM 异步调用削峰；pgvector 迁移到独立 Qdrant；MySQL 按业务模块分库。

不改什么：Spring Boot 仍是单体不做微服务拆分，Redis 架构不动。

难点：LLM 异步化后，前端要从 SSE 改成轮询或 WebSocket，是 breaking change。这个改动成本不低，所以不到触发条件不做。

第三阶段：2000 → 几千人

触发条件：单个模块的发布频率互相干扰；某一模块（如 Agent 执行）资源消耗远超其他模块。

改什么：按模块拆微服务（Agent 执行、知识库、对话历史各自独立）；引入 API Gateway 替代 Nginx 路由；Redis 换集群模式。

不改什么：MySQL 主从架构不动，Qdrant 不动，K8s 编排层不动。

难点：微服务拆分带来分布式事务问题，需要引入 Saga 或 TCC 模式。第 04 讲定的“跨模块走 Service 接口”在这里发挥作用——拆分时只改调用方式，接口签名不变。

三个阶段的对比总览：

	一期（当前）	二期	三期
规模	50 人	500 人	2000 人+
部署	单副本	多副本	微服务
数据库	MySQL 单机	主从分离	分库
向量库	pgvector	pgvector	Qdrant
LLM 调用	同步 SSE	异步 MQ	异步 MQ
缓存	Redis 单节点	Redis 单节点	Redis 集群
最大难点	SSE 超时	LLM 异步化	分布式事务

我们用一张图来总结下：



这张表就是你的扩展路线图。现在不做任何一步，但每一步的触发条件、改什么、难点在哪，全部清楚。而且你应该注意到了，第 04 讲的每个设计决策都在为这条路径留口子：**模块化单体让第三阶段的微服务拆分成本可控，接口隔离让调用方式可以平滑切换，会话存 Redis 让第一阶段的多副本不需要改代码**。

架构设计的功力不在于现在做多复杂，在于现在做的每个决策都不堵未来的路。

数据模型概览

核心表有哪些、关系怎么样，先列出来。具体字段设计等后面每个模块开发时再定，现在定太细了后面一定会改。

> 基于 Hify 的功能范围（模型管理、Agent、对话、工作流、知识库、MCP 工具），帮我梳理核心数据表和它们之间的关系。只要表名和关系，不展开字段。

Claude Code 给了一份完整的梳理，我确认后定稿。按功能域分组：

模型管理：model_provider（提供商）→ model（具体模型），一对多。一个 OpenAI 下有 GPT-4o、GPT-3.5 多个模型。

知识库：knowledge_base → document → document_chunk，一对多链。
document_chunk 是向量化的最小单位，存在 pgvector 里。

工具：tool，独立表，存 MCP 工具定义。

Agent：agent 是关联最多的表，agent → model（多对一，选一个模型），
agent ↔ knowledge_base（多对多，通过 agent_knowledge_base），agent ↔ tool（多对多，通过 agent_tool）。

工作流：workflow → workflow_node，一对多。workflow_node 里的 LLM 节点也关联 model。

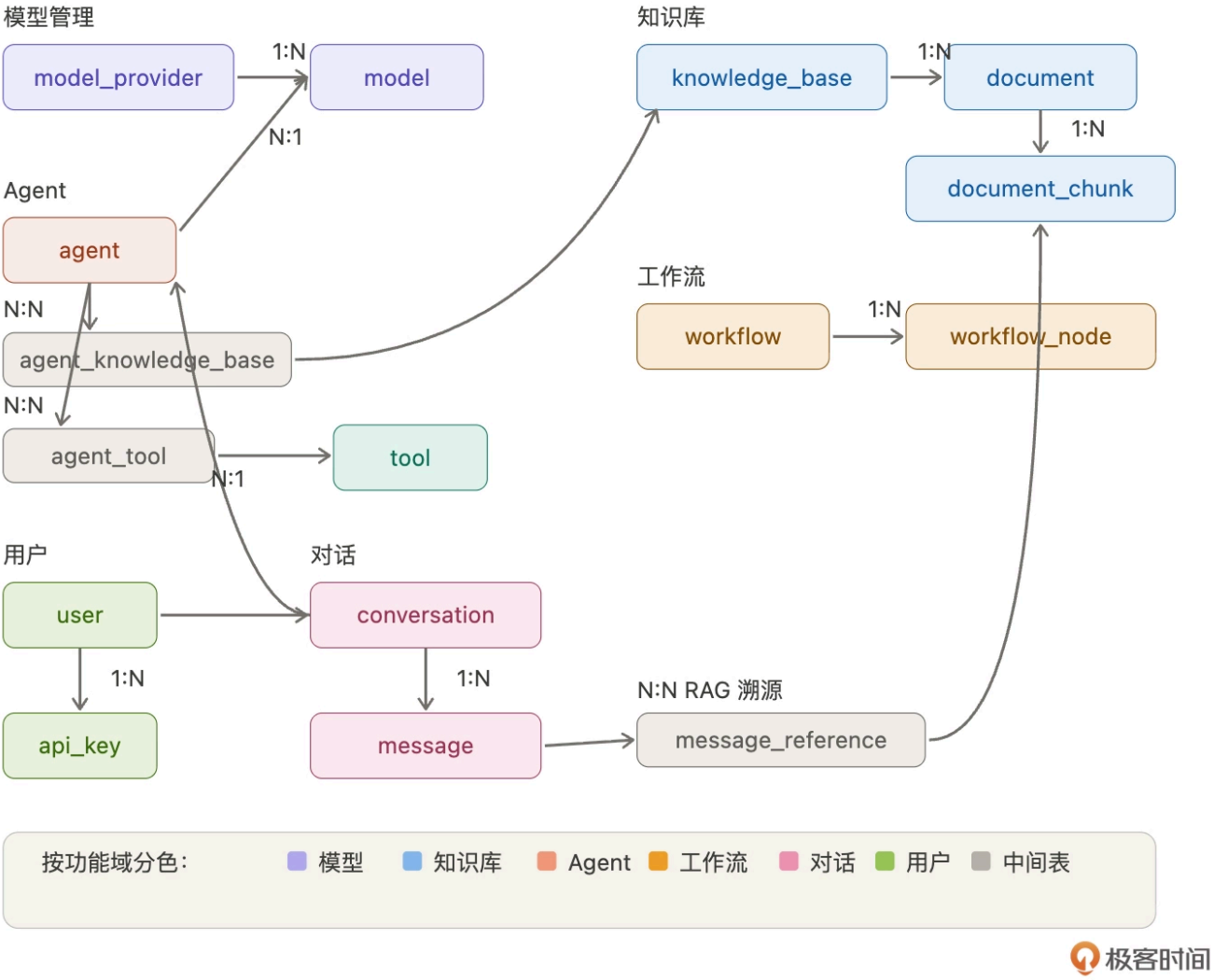
对话：conversation → message，一对多。conversation 关联 agent 或 workflow（对话绑定的执行对象）。message → document_chunk 多对多（通过 message_reference，做 RAG 溯源——回答引用了哪些知识库片段）。

用户：user 和 api_key。user 发起 conversation，api_key 用于 API 发布调用。

关系汇总：

关系	类型
model_provider → model	一对多
knowledge_base → document → document_chunk	一对多链
agent ↔ knowledge_base	多对多
agent ↔ tool	多对多
agent → model	多对一
workflow → workflow_node	一对多
conversation → agent / workflow	多对一
conversation → message	一对多
message → document_chunk	多对多（RAG 溯源）

总共约 16 张表。看着多，但按功能域分组后每组就两三张，结构清晰。



说实话，Claude Code 设计的这个表结构比我想的完整得多，比我设计的好，设计的快。AI 很适合这种模板化的任务。

这里想说，你要学会的一个点是：**你要清晰地知道 AI 能干什么，干什么干的好，而你要做什么。我们跟 AI 不是敌人，不是竞争者，而是合作伙伴。我们要学会的是和它一起协作，释放我们自己最大的效率，从而给我们自己带来更大的现实收益。**

有一个设计决策值得说明：message_reference 这张表，它记录的是“这条 AI 回复引用了知识库的哪些片段”。很多人做 RAG 不存这个关系，回复完就丢掉了检索结果。但存下来有两个好处：一是用户可以看到“AI 的回答依据是什么”，增加可信度；二是后续优化 RAG 效果时，可以回溯分析检索质量。一张中间表，成本很低，价值不小。

数据库性能规范

表结构后面再定，但数据库层面的规范现在就要定，后面 Claude Code 每次建表、写查询都要遵守。

> Hify 用 MySQL 8.x + pgvector。帮我定义数据库层面的性能规范，覆盖：索引设计原则、大表预判和应对策略、分页查询注意事项、通用字段约定。要求具体到 AI 建表时能直接执行。

Claude Code 给了一份非常详细的规范，包含了具体的建表示例。我逐块确认后定稿。

通用字段约定

所有表必须包含四个公共字段：

```
复制代码
id          BIGINT          NOT NULL AUTO_INCREMENT PRIMARY KEY,
created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRE
deleted     TINYINT(1)      NOT NULL DEFAULT 0
```

几条硬规矩：主键用自增 BIGINT，禁止 UUID（索引性能差）。禁止用 NULL，业务空值用空字符串或 0 代替。金额和 Token 用量用 BIGINT 存最小精度，不用 DECIMAL。枚举字段用 VARCHAR(32)，不用 MySQL ENUM 类型（ENUM 加值要改表结构）。你看，这是不是就是我们进入团队要学习的团队规范。

索引设计原则

Claude Code 给了五条具体规则，每条带示例。

第一，逻辑删除字段必须加进索引。几乎所有查询都带 `deleted = 0`，不加进索引等于索引白建。

```
复制代码
-- ✅ 正确
INDEX idx_agent_user (user_id, deleted)
```

```
-- ❌ 不够
INDEX idx_agent_user (user_id)
```

第二，组合索引等值列在前，范围列在后。这是 MySQL 索引的基本原则，但不提醒 Claude Code，它自己写查询时经常不遵守。

复制代码

```
INDEX idx_message_conv_time (conversation_id, created_at)
```

第三，多对多关联表两个方向都要索引。agent_tool 表按 agent_id 查和按 tool_id 查都是高频操作，只建一个方向的索引，另一个方向就全表扫描。

复制代码

```
PRIMARY KEY (agent_id, knowledge_base_id),
INDEX idx_kb_agent (knowledge_base_id)    -- 反向查询索引
```

第四，唯一约束用 UNIQUE INDEX，不要只在代码层校验。并发场景下代码校验有竞态问题，数据库约束才是最后防线。

第五，禁止在大文本字段建索引。content、prompt 这类 TEXT 字段不能建索引，需要全文搜索的场景后续引入 ES。

大表预判

Hify 中增长最快的两张表：

message 表：每次对话产生 2–N 条记录，50 人每天几百到上千条。

应对：建好时间范围索引（conversation_id + created_at），预留按时间归档的能力。一期建好索引够用半年。

document_chunk 表：知识库分块数据，100 篇文档可能产生 5000+ 行。

应对：向量数据存 pgvector，MySQL 只存元数据和 vector_id（指向 pgvector 的记录），不在 MySQL 里存向量本身。

其他表（provider、agent、workflow）都是配置数据，增长慢，不需要特别关注。

分页查询规范

复制代码

```
-- ❌ 禁止 OFFSET 深分页，百万行时极慢
SELECT * FROM message ORDER BY id DESC LIMIT 20 OFFSET 100000;

-- ✅ 用游标分页
SELECT * FROM message
WHERE conversation_id = ?
  AND id < #{lastId}
  AND deleted = 0
ORDER BY id DESC
LIMIT 20;
```

一期数据量小，OFFSET 够用。但规范先定好，Claude Code 知道用游标分页，后面数据量上来了不用改代码。管理后台必须用 OFFSET 的场景，限制最大页数（超过 10000 条直接提示缩小查询范围）。

还有一条：COUNT 查询单独处理，列表页只在第一页返回 total，翻页不重复查。这个细节 Claude Code 不说你很容易忽略，但它对大表的查询性能影响很大。

pgvector 规范

向量数据单独存 PostgreSQL，和 MySQL 业务数据分离：

复制代码

```
CREATE TABLE vector_chunk (
    id          BIGSERIAL PRIMARY KEY,
    chunk_id    BIGINT       NOT NULL,    -- 对应 MySQL document_chunk.id
    embedding   vector(1536) NOT NULL,
```

```
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- 必须建 HNSW 索引，否则全表扫描
CREATE INDEX idx_embedding_hnsw
ON vector_chunk
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

两条硬规矩：必须建 HNSW 索引（一期数据量小感知不到差异，但数据量上来后，没索引检索会从毫秒级变成秒级）。检索时必须加 LIMIT，禁止不加 LIMIT 的向量查询（全量排序会拖垮数据库）。

这些规范看起来是数据库基础知识，但你不写进 CLAUDE.md，Claude Code 建表时就不会自动加 deleted 进索引、不会考虑分页性能、不会预判大表问题、不会给 pgvector 建 HNSW 索引。写了，后面每一次建表、每一次写查询都自动遵守。

如果有心，你可以从上面的这个问题以及它的答案中，知道设计数据库表应该注意什么。我觉得这就是我们这节课的价值。**问了问题，辨别答案，学习答案**——这才是把 AI 融入工作的核心思路。

写进 CLAUDE.md

这一讲的所有决策写进 CLAUDE.md：

复制代码

```
### 部署架构
生产环境：Docker + K8s
- 前端：Nginx 托管静态文件 + API 反向代理（proxy_buffering off）
- 后端：Spring Boot，K8s Deployment（一期单副本）
- 数据库：MySQL 8.x（外部服务）
- 缓存：Redis 7.x（外部服务）
- 向量库：PostgreSQL + pgvector（外部服务）
- 本地开发：java -jar + npm run dev，start.sh 一键启动

### 缓存策略
- Provider/Agent 配置：Redis Cache-Aside，TTL 30min
- 对话上下文：Redis，TTL 2h
```

```
- 对话消息、知识库文档：不缓存，走数据库
- LLM 响应：不缓存

### 数据库规范
通用字段：
- 主键 id BIGINT 自增，禁止 UUID
- 时间字段 created_at / updated_at，DATETIME(3)
- 逻辑删除 deleted TINYINT(1)
- 禁止 NULL，空值用空字符串或 0
- 枚举用 VARCHAR(32)，不用 MySQL ENUM

索引规则：
- 命名 idx_{表名}_{字段名}
- 逻辑删除字段必须加进组合索引
- 组合索引等值列在前，范围列在后
- 多对多关联表两个方向都要索引
- 唯一约束用 UNIQUE INDEX，不只在代码层校验
- 禁止在 TEXT/BLOB 字段建索引
- 不建数据库级外键约束，应用层维护

分页规则：
- 默认用游标分页（WHERE id < lastId ORDER BY id DESC LIMIT N）
- OFFSET 分页限制最大 10000 条
- COUNT 只在第一页查，翻页不重复查

大表预判：
- message：增长最快，必须建（conversation_id, created_at）索引
- document_chunk：MySQL 只存元数据，向量存 pgvector

pgvector 规范：
- 向量表建在 PostgreSQL，维度固定 1536
- 必须建 HNSW 索引
- 检索必须加 LIMIT，禁止全量排序

### 扩展路径
一期单副本 → 多副本 + 主从分离（500人）
→ MQ 异步 + Qdrant（2000人）→ 微服务拆分 + Redis 集群（几千人）
触发条件驱动，条件不到不动
```

到这里，CLAUDE.md 包含了项目概述（第 03 讲）、应用架构与代码组织（第 04 讲）、部署架构与数据库规范（本讲）。Claude Code 对 Hify 从代码怎么写到系统怎么跑都有了完整的认知。

总结

这一讲做了四件事：设计了当前部署架构，预判了性能瓶颈，规划了扩展路径，定义了数据模型和数据库规范。

最有价值的是**性能瓶颈预判**。大部分工程师不会提前想这个，觉得一期先跑起来再说。但你作为架构师，脑子里要有一张地图：瓶颈在哪、什么时候会触发、到时候改什么。有了这张地图，你对系统的掌控力完全不一样。而且这张地图不需要你自己从零画，让 Claude Code 基于你的架构分析，你判断它说得对不对就行。

另一个值得强调的点是**数据库规范**。这些内容（索引原则、分页注意事项、大表预判）对有经验的工程师来说是常识，但对 Claude Code 来说不是。你不写，它不会自动遵守。这些规范现在花 10 分钟定好，后面每一次建表、每一次写查询都受益。

上一讲解决了代码怎么组织，这一讲解决了系统怎么跑。两讲合起来就是 Hify 的完整架构设计——从模块划分到部署形态到扩展路径到数据规范，一条线拉通。下一讲我们将进入规范体系搭建，手把手带你写完整的 CLAUDE.md，把前面所有决策结构化地沉淀下来。

思考题

让 Claude Code 帮你分析你当前项目的性能瓶颈。把你的部署架构描述给它，用了什么数据库、什么缓存、多大用户量、主要压力在哪，然后问“这个架构的性能瓶颈可能在哪”，看看它指出的问题里有没有你之前没想到的。最后记得把它的分析和你的判断写在留言区。

期待你的分享。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (7)

最新　精选



-m

2026-04-06 来自广东

老师，你好。我不知道您后面有没有对claude md 做优化，我个人觉得，还有很多优化空间。md文件是常驻上下文的，并且是全量加载的，md文件写 基本的项目描述，代码组织结构 等基本信息可以了，比如 数据库、项目架构、宪法等规范，可以在claude md 去引用相关文件，或者将这些知识沉淀为skill，对于节省大模型上下文，是很有帮助的。

> 作者回复: 会的，最后会把claude.md 和skill做一个完整的拆分讲解。因为课程是一步一步来的。我希望从最简单的claude.md 开始讲，降低大家的学习理解成本。在合适的时候引入skills 等高级工具。

10



心明眼亮

2026-05-27 来自江苏

都用pgvector了，为什么不直接用pg还要用mysql，请问是怎么考虑的?

> 作者回复: 最开始想的是用的mysql来存储关系数据。然后在选型向量数据的时候才选型pg。从合理性来看，直接用pg是合理的。一开始没想到关系检索和向量检索可以一把用pg搞定。感谢提醒，学到了～🌹

3



cricket1981

2026-04-12 来自上海

这样一股脑东西全塞进 CLAUDE_CODE.md 文件是不是不太好，整一个数据库设计规范文件，在CLAUDE_CODE.md中引用它是不是更好?

> 作者回复: 作为一个小项目，CLAUDE_CODE.md 能满足很多需求。项目变大了就要拆了，我在另外一个回答里面回答了这个问题。

1



永不言弃

2026-04-09 来自广东

我有一个疑问，cc 有一个 /init 的命令 ，好像自动生成的内容不会写这么细致

作者回复: 只有一个 /init 肯定没有这个效果，你要有输入才能这么详细。要有讨论上下文，或者提示。比如新项目就得跟他说我们要做什么，可以看看06讲。所以这门课，我们一直强调输入，也就是交互。不是让它自己跑。

共 3 条评论>

1



skytechfyh

2026-05-23 来自江苏

老师后面的代码开发，是sonnect 4.6 还是 ops4.7？ 另外我按照老师同样的提示词，输出的结果有些不一样，比如：本节课的第一个prompt，我这边ops4.7 给出的答案，一直在纠结k8s的问题，纠结前面几节明明说了是单体服务，使用docker compose的问题，以及大多数的回答都是基于k8s的，在这里看，这个提示词感觉有点误导

作者回复: 我用的是：sonnect 4.6 。
课程内的思路：部署的最终目标是k8s部署，只是也提供了二进制部署的模式。
单体服务对应的是微服务，也就是没拆那么多微服务。单体服务和k8s不冲突。
k8s 部署和二进制部署是支持的两种形态



王洪

2026-05-12 来自四川

思路无问题，但是MySQL 8.x + pgvector 存在不兼容

作者回复: 这个是有报错吗？ 可以发出来一起看看。

星辞Sting

2026-04-26 来自浙江

看到这里才发现，自己用AI CODING 和老师用 差距真的很大。我开始站在开发的角度去想 AI coding 能怎么样帮我吞吐更多的需求， 虽然Vlbe Coding直接能做需求，但是这种不是指数级的，。对于各个企业落地难，存量系统的各种梳理和理解以及坑 是一个难点，前期这个梳理的有多好，中后期就会是指数飞跃。 但是老板们都追求短期就要出效果 也是很难

作者回复: 我觉得吧，熟能生巧。用多了就好了。多看看别人怎么做的，然后自己多练，就好了。我感觉我也没啥特殊技巧。就写多了。

另外我在5月份会上一门“老项目改造”的课，专门讲存量系统的改造。到时候可以看看效果。

“老板们都追求短期就要出效果 也是很难”。其实AI 可以解决这个问题的。

共 2 条评论 >